Sistemas Operacionais

Resumo completo para a prova

Fundamentos • Processos • Threads • Comunicação • Sincronização • Escalonamento • Deadlocks

Conceitos essenciais em **azul e negrito**, para destacar no caderno. Texto de apoio em preto.

Escopo: tópicos informados, guia da avaliação (capítulos 2 a 13), Lista 1 e materiais de aula. Este resumo prioriza definições, diferenças, mecanismos e regras de resolução.

Como estudar

Leia cada bloco e tente explicá-lo sem consultar. Em sincronização, simule as operações; em escalonamento, desenhe a linha do tempo; em deadlocks, identifique recursos e dependências.

Aula 1 - Fundamentos: revisão

SO gerencia recursos, controla a execução e oferece abstrações. Gerenciar: escolher quem usa CPU/memória/dispositivos. Controlar: verificar acessos e tratar operações indevidas. Abstrair: oferecer arquivos em vez de exigir controle direto dos setores do disco. Objetivos: conveniência, eficiência e proteção.

Lote reduz preparação entre trabalhos; multiprogramação aproveita a espera por E/S; tempo compartilhado melhora a interação. Concorrência é progresso sobreposto; paralelismo é execução simultânea.

Modo usuário restringe operações; modo kernel permite executar as rotinas privilegiadas do núcleo. Syscall transfere o controle para uma rotina autorizada que valida a solicitação. Ela não concede liberdade privilegiada ao aplicativo. Interrupção de hardware comunica eventos como temporizador e término de E/S.

Troca de modo altera privilégio; troca de contexto altera a tarefa em execução. Uma syscall pode retornar ao mesmo processo sem que outro execute.

Monolítico: muitos serviços no kernel. Micronúcleo: núcleo reduzido e serviços externos, com maior isolamento e comunicação adicional. Camadas: níveis de responsabilidades. Híbrido: combinação de decisões. Shell interpreta comandos; não é o kernel.

Aula 2 - Processos

Programa é passivo; processo é uma instância em execução. Abrir o mesmo programa duas vezes pode criar processos com PID, memória e contexto próprios. O programa original continua existindo.

Memória: código contém instruções; dados contêm globais/estáticas; heap atende alocação dinâmica; pilha mantém informações das chamadas. Processos podem compartilhar regiões explicitamente, mas suas variáveis não são automaticamente a mesma variável.

PCB reúne informações de gerenciamento, não toda a memória do processo. Mantém ou referencia PID, estado, PC, registradores, memória, prioridade e recursos. PC informa onde continuar; registradores e contexto de memória permitem continuar com valores e mapeamentos corretos.

Estados

Pronto espera CPU; bloqueado espera evento.

• Novo → pronto: admissão após criação.

• Pronto → executando: despacho.

• Executando → pronto: preempção, como fim do quantum.

• Executando → bloqueado: espera necessária por E/S/recurso.

• Bloqueado → pronto: evento concluído.

• Executando → terminado: conclusão/encerramento.

O desbloqueio não concede CPU automaticamente. Prontos e bloqueados são organizados em filas/listas, possivelmente várias. A fila de prontos não precisa seguir FIFO.

Criação e término

fork cria; exec substitui; wait aguarda/recolhe término.

fork retorna 0 no filho, PID do filho no pai e −1 em caso de falha. Pai e filho continuam após a chamada. Têm espaços de endereçamento separados; copy-on-write pode compartilhar páginas até uma escrita. Descritores herdados podem referenciar a mesma descrição de arquivo aberto: separação de memória não implica independência de todos os recursos.

exec substitui a imagem do processo pelo novo programa, mantendo PID; em caso de sucesso não retorna ao código antigo. exit encerra o processo. wait pode bloquear até um filho terminar e recolhe seu status.

Zumbi já terminou e aguarda coleta do status; órfão perdeu o pai e pode continuar executando. Encerrar o pai não encerra automaticamente todos os filhos.

Exemplo: três fork incondicionais executados por todos, sem falhas, resultam em $2^3=8$ processos totais, dos quais 7 novos. Um print depois deles executa 8 vezes; prints dentro do laço exigem contagem por etapa. Não aplique 2^N se houver condições que impeçam alguns processos de executar os forks.

Aula 3 - Threads e comunicação

Processo agrupa recursos; thread é uma linha de execução dentro dele. Threads compartilham código, globais, heap e arquivos; cada uma tem PC, registradores, pilha, identificador e estado próprios. Sua pilha é individual, mas está no espaço compartilhado e pode ser acessada por outra thread mediante referência válida.

Benefícios: resposta, economia de criação/comunicação e paralelismo em vários núcleos quando o modelo permite. Isolamento entre processos é maior; compartilhar memória facilita comunicação e também amplia efeitos de erros. Uma falha grave de uma thread pode afetar o processo inteiro, mas nem toda falha de thread necessariamente o encerra.

Modelo	Paralelismo das threads do processo	Bloqueio	Gestão
N:1	Não usa vários núcleos simultaneamente	Syscall bloqueante pode bloquear o conjunto	Leve, em biblioteca
1:1	Possível em vários núcleos	Bloqueio de uma não exige bloqueio das demais	Mais recursos de kernel
N:M	Limitado pelas threads de kernel disponíveis	Outras podem continuar usando threads de kernel disponíveis	Implementação complexa

Bibliotecas: POSIX pthreads (pthread_create cria, pthread_join espera término); APIs de threads do Windows; interfaces de threads de Java. São interfaces de programação, não garantias universais sobre mapeamento.

Processos cooperativos trocam/compartilham informações, permitindo divisão do trabalho e compartilhamento de dados, com riscos de corrida e deadlock. Processos independentes, no modelo conceitual, não dependem dos dados uns dos outros.

Memória compartilhada exige coordenação dos acessos; mensagens não eliminam toda necessidade de sincronização da aplicação. Memória compartilhada evita cópias intermediárias após o mapeamento e costuma ser eficiente localmente. Mensagens mantêm fronteiras e maior separação dos espaços; funcionam localmente e em rede. Desempenho depende da implementação.

Pipe: fluxo de bytes, normalmente unidirecional no modelo do curso. Pipeline encadeia saída de um processo à entrada de outro. Fila: guarda mensagens até recebimento, com limites de capacidade. Cliente-servidor: cliente solicita, servidor atende; pode utilizar diferentes mecanismos de comunicação.

Operação bloqueante espera uma condição; não bloqueante retorna sem esperar indefinidamente. Capacidade zero exige encontro entre emissor/receptor para transferência; capacidade finita pode bloquear envios quando cheia. Mensagens mantêm unidades; fluxo de bytes exige definir como separar os dados.

Aula 4 - Sincronização

Corrida: o resultado depende de intercalações não coordenadas de acessos compartilhados, com pelo menos uma alteração. Basta concorrência, mesmo em um núcleo.

contador++ envolve ler, calcular e escrever. Com contador=0: A lê 0, B lê 0, A grava 1, B grava 1. Resultado 1, embora houvesse dois incrementos. Região crítica é o trecho que exige coordenação para preservar a consistência do recurso compartilhado.

Exclusão mútua impede entradas simultâneas; progresso impede adiamento indevido; espera limitada limita ultrapassagens. Os requisitos se referem às seções que disputam o mesmo recurso/protocolo, não a todas as seções críticas do sistema.

Soluções e limitações

• Trava ingênua: testar livre e depois marcar ocupado deixa uma janela de corrida.

• Alternância estrita: obriga esperar a vez mesmo se o outro não quiser entrar; pode violar progresso.

• Peterson: interesse e turno coordenam duas tarefas sob hipóteses de atomicidade/ordenação; usa espera ocupada.

• TSL/Test-and-Set: teste e alteração atômicos; pode construir spinlock.

• Desativar interrupções apenas localmente não protege contra outro núcleo.

• sleep/wakeup: suspender/acordar exige protocolo correto para não perder um aviso entre o teste e a suspensão.

• **Espera ocupada consome CPU verificando uma condição; bloqueio suspende a tarefa.** Spinlocks podem ser adequados para esperas muito curtas em certos contextos de kernel.

Bernstein: independência requer $W(A) \cap R(B) = \emptyset$, $R(A) \cap W(B) = \emptyset$ e $W(A) \cap W(B) = \emptyset$. R é o conjunto lido, W o escrito.

Mutex e semáforo

Mutex tem posse e protege exclusão; semáforo conta permissões e também sinaliza eventos. Um semáforo binário pode servir à exclusão, mas não tem necessariamente as garantias de posse de um mutex. Herança de prioridade e detecção de erros dependem da implementação/configuração.

wait/down/P: obtém uma permissão ou espera. signal/up/V: fornece uma permissão, podendo acordar uma tarefa. Operações atômicas. Inicialização: 1 para uma permissão exclusiva; N para N recursos; 0 para aguardar evento.

Semáforo=3 e cinco waits de tarefas diferentes: até 2 ficam bloqueadas se todos os waits vierem primeiro. Depois de dois signals, nenhuma continua esperando uma permissão nessa sequência, embora acordada não signifique já executando. Não use apenas uma subtração para deduzir o pico de bloqueio.

Produtor-consumidor

Exclusão e disponibilidade são problemas diferentes. Para buffer inicialmente vazio de capacidade N: mutex=1, vagas=N, itens=0.

```
Produtor:          Consumidor:
wait(vagas)          wait(itens)
wait(mutex)          wait(mutex)
inserir            retirar
signal(mutex)        signal(mutex)
signal(itens)        signal(vagas)
```

Espere pela disponibilidade antes de adquirir o mutex. Se o produtor segurar mutex e esperar vagas com buffer cheio, pode impedir o consumidor de retirar e liberar uma vaga.

Leitores-escritores: vários leitores podem acessar juntos, mas escrita requer exclusividade. Priorizar continuamente leitores pode deixar escritores esperando indefinidamente; políticas justas buscam evitar isso.

Filósofos: cada tarefa precisa de dois garfos exclusivos. Se todos seguram um e esperam o próximo, surge ciclo. Soluções: ordem global dos garfos, no máximo N−1 tentando adquiri-los ou token/saleiro para aquisição do par. **Evitar deadlock não garante ausência de starvation.**

Monitor e condições

Monitor encapsula dados e operações com exclusão mútua automática; variável de condição permite esperar pelo estado necessário. Reduz esquecimentos de aquisição/liberação, mas ainda pode conter protocolos errados.

```
adquirir mutex
while buffer vazio:
    esperar(condicao, mutex)
retirar item
liberar mutex
```

Esperar libera atomicamente o mutex e suspende; antes do retorno, readquire o mutex. **Use while para testar novamente: acordar não garante que a condição permaneça verdadeira.** Sinal de condição não acumula crédito como signal de semáforo. Hoare: sinalização transfere imediatamente o monitor ao acordado; Mesa: sinalizador continua e acordado disputa a entrada depois.

Inversão de prioridade: tarefa importante espera lock de tarefa menos prioritária, atrasada por tarefas intermediárias. Herança de prioridade pode elevar temporariamente quem segura o lock.

Aula 5 - Escalonamento

Escolha apenas entre processos que já chegaram e estão prontos. Preemptivo pode retirar CPU; não preemptivo deixa a tarefa continuar até bloquear, terminar ou ceder.

Algoritmo	Regra	Atenção
FCFS	Ordem de chegada	Efeito comboio: tarefa longa atrasa curtas
SJF	Menor próxima rajada	Não interrompe a atual; duração precisa ser conhecida/estimada
SRTF	Menor tempo restante	Versão preemptiva; reavaliar nas chegadas
Prioridade	Maior prioridade conforme a escala	Pode ser preemptivo ou não; risco de starvation
Round Robin	Fila de prontos e fatias até quantum	Quem não termina volta ao fim da fila

SJF/SRTF podem adiar tarefas longas continuamente. **Aging aumenta a prioridade conforme a espera.** RR com quantum muito grande aproxima FCFS; quantum pequeno melhora oportunidades de atendimento, mas aumenta overhead. Não há garantia de que sempre reduza espera média.

Critérios: maximizar utilização/vazão; minimizar espera, retorno e resposta; em tempo real, cumprir prazos. CPU-bound faz uso intenso de CPU; I/O-bound tem frequentes esperas por E/S.

Retorno = conclusão − chegada. Resposta = primeiro início − chegada. Espera = retorno − rajada, esta última para uma única rajada sem E/S. Em cenários gerais, espera é a soma dos períodos na fila de prontos.

Exemplo: A chega 0 com duração 4; B chega 1 com duração 2.

• FCFS/SJF não preemptivo: A 0-4; B 4-6. Esperas A=0, B=3; média=1,5.

• SRTF: A 0-1; B 1-3; A 3-6. Esperas A=2, B=0; média=1.

• RR q=2: A 0-2; B 2-4; A 4-6. Esperas A=2, B=1; média=1,5. Respostas A=0, B=1.

Para resolver: escreva chegada/rajada/prioridade; defina empates; registre fila e tempo restante; desenhe intervalos; anote conclusão/primeiro início; calcule métricas. Se ninguém estiver pronto, marque CPU ociosa. Considere overhead apenas quando informado.

Complementos: loteria distribui chances por bilhetes, sem garantir prazo individual; múltiplas filas separam classes; feedback permite mudar entre filas conforme comportamento; sistemas de tempo real priorizam requisitos de prazo.

Aula 6 - Deadlocks

Deadlock é espera sem progresso entre tarefas dependentes; starvation é adiamento indefinido de uma tarefa enquanto outras podem avançar; livelock é atividade sem progresso útil.

Quatro condições necessárias de Coffman:

1. **Exclusão mútua:** recurso não compartilhável simultaneamente.

2. **Posse e espera:** segura recurso enquanto espera outro.

3. **Não preempção de recursos:** não podem ser retirados à força no modelo considerado.

4. **Espera circular:** cadeia fechada de dependências.

Não preempção de recursos não significa que a CPU precise ter escalonamento não preemptivo.

Exemplo: A segura R1 e pede R2; B segura R2 e pede R1. No grafo: R1→A→R2→B→R1. **Processo→recurso é pedido; recurso→processo é posse.** Com uma instância por tipo, ciclo implica deadlock; com várias, não basta ver um ciclo.

Estratégia	Como age
Prevenção	Impede alguma condição necessária
Impedimento/evitação	Analisa concessões para manter estado seguro
Detecção	Procura deadlock já existente
Recuperação	Encerra tarefas, retira recursos quando viável ou usa rollback

Prevenção: solicitar tudo junto ataca posse e espera, mas pode desperdiçar recursos; ordem global ataca espera circular; compartilhar recursos quando possível ataca exclusão; retirar recursos exige viabilidade, não serve universalmente. Avestruz: não tratar genericamente, aceitando risco/custo de intervenção.

Estado seguro e algoritmo do banqueiro

Seguro: existe uma ordem em que todos podem terminar, respeitadas suas demandas máximas. Inseguro não significa deadlock já ocorrido.

Necessidade = Máximo − Alocado, por tipo de recurso. Work=Disponível. Procure processo não finalizado com Necessidade≤Work em todos os tipos; simule término e faça Work += Alocado; repita. Se todos terminam, há sequência segura. Se não encontra candidato antes de todos terminarem, o teste não demonstrou segurança.

Exemplo com um tipo: disponível=1; A alocado=1/máximo=2; B alocado=1/máximo=3. Necessidades A=1, B=2. A pode terminar; Work passa a 2; B termina; Work passa a 3. Seguro: A→B.

Antes de conceder pedido: pedido≤necessidade, pedido≤disponível, simulação da concessão e teste de segurança. **Na simulação de término, some a alocação original do processo; não some sua necessidade outra vez.** Detecção usa pedidos pendentes e alocação conforme o modelo, enquanto segurança considera demandas máximas futuras.

Revisão final: o que não confundir

Programa / processo: Programa contém instruções; processo é sua instância em execução.

Pronto / bloqueado: Pronto espera CPU; bloqueado espera um evento necessário.

fork / exec / wait: Criar filho / substituir imagem / aguardar e recolher término.

Zumbi / órfão: Terminou sem coleta de status / perdeu o pai.

Modo / contexto: Privilégio da execução / estado de execução da tarefa.

Concorrência / paralelismo: Progresso sobreposto / execução simultânea.

Mutex / semáforo: Trava com posse / permissões e sinalização.

Exclusão / disponibilidade: Proteger acesso / verificar se a operação pode ocorrer.

SJF / SRTF: Menor próxima rajada sem preempção / menor tempo restante com preempção.

Resposta / retorno: Primeiro início menos chegada / conclusão menos chegada.

Deadlock / starvation / livelock: Impasse por dependências / adiamento indefinido / atividade sem progresso útil.

Prevenção / impedimento: Quebrar condição necessária / manter estado seguro ao conceder.

Inseguro / deadlock: Sem garantia de sequência segura / impasse existente.

Fontes e uso

Síntese dos materiais fornecidos: Guia_Prova_1; slides das Aulas 2 a 11 disponíveis; livro de Carlos Maziero, capítulos 2 a 13; laboratórios 01 a 03; Lista 1 com 17 questões; prova de 2024. Exemplos numéricos didáticos não identificados como questões reais foram elaborados para revisão. Memória avançada, arquivos e virtualização ficam fora deste escopo de prova indicado.